

Supplement to “LeetDecoding: A PyTorch Library and Benchmark for Exponentially Decaying Causal Linear Attention”

APPENDIX A HOW TO USE LeetDecoding

We introduce how LeetDecoding is designed to serve various LLM practitioners. At the end of this section, we as well provide a snippet to demonstrate the practical usage of LeetDecoding.

For **application developers** using linear transformers to achieve supreme inference efficiency, LeetDecoding can be integrated into transformer-based models through its API. Users can directly replace the original attention computation interface with LeetDecoding for attention calculation. The input format of this library is consistent with the most popular computation libraries such as HuggingFace Transformers⁴ or FlashAttention⁵.

For **researchers** in the community of machine learning and data science, LeetDecoding can serve as a benchmark for profiling the newly proposed linear attention algorithms, as shown in Listing 1. LeetDecoding is intentionally designed for easy extension, and new computation methods can be implemented therein through creating the corresponding method class and subsequently registering the class in LeetDecoding.

Listing 1: An example of how to use LeetDecoding Library to benchmark a new implementation.

```
1 from efficient_linear_decoding.test_method import benchmark_method
2
3 def attn(q, k, v, gamma):
4     # customize the linear attention computation method here.
5     pass
6
7 # Example usage of the benchmark_method() function:
8 # Note: The values below should be replaced with the most suitable values.
9
10 benchmark_method(method=attn, device='cuda', dtype='float32', batch_size=16,
11                  is_weight_decay=True, gamma=gamma)
```

Listing 2: Usage of the causal_linear_decoder function, requiring only one line for regular causal linear attention with binary masks or the one with exponentially decaying causal masks.

```
1 from efficient_linear_decoding import causal_linear_decoder
2
3 batch_size, heads, seqlen, rank, dim = 1, 32, 4096, 128, 256
4 B = torch.randn((batch_size, heads, seqlen, rank), device='cuda')
5 C = torch.randn((batch_size, heads, seqlen, rank), device='cuda')
6 V = torch.randn((batch_size, heads, seqlen, dim), device='cuda')
7 gamma = torch.full((heads, 1), 0.9, device='cuda')
8
9 # If the user intends to apply a 0-1 causal mask.
10 output = causal_linear_decoder(B, C, V)
11
12 # If the user intends to apply an exponentially decaying mask.
13 output = causal_linear_decoder(B, C, V, is_mask_weight=True, gamma=gamma)
14
15 # If the user intends to apply a specified method in the library.
16 output = causal_linear_decoder(B, C, V, attn_method='block-based')
17
18 # If the user intends to manually tune the GPU hyperparameters.
19 custom_configs = [
20     triton.Config({'BLOCK': 64, 'BLOCK_MODEL': 32}, num_warps=4, num_stages=2),
21     triton.Config({'BLOCK': 128, 'BLOCK_MODEL': 64}, num_warps=8, num_stages=3),
22 ]
23 output = causal_linear_decoder(B, C, V, attn_method='lightningAttention2',
24                                autotune_configs=custom_configs)
```

To demonstrate the simplicity and the convenience of the coding style in LeetDecoding, we provide another snippet to show how LeetDecoding solves synthetic problems efficiently by a few lines of code. Specifically, Listing 2 gives four examples to illustrate how to use the LeetDecoding library, ranging from the passing of input arguments, to the specification of the mask type and the computation method. For the library user’s convenience, the causal_linear_decoder(B, C, V) function can automatically specify the computation method and the associated settings. Specifically for methods implemented with the

⁴See <https://github.com/huggingface/transformers>.

⁵See <https://github.com/Dao-AI-Lab/flash-attention>.

OpenAI Triton language (**Lightning Attention-2**, which allows low-level GPU configurations), we maintain a built-in table based on the numerical experiments we performed in Section VI. Under the hardware and the input types (the batch size and the sequence length) we have investigated, the table directly returns the empirically optimal settings. For unseen hardware and settings, advanced users can conveniently compare multiple sets of GPU hyperparameters and specify optimal configurations on their own, through an argument `autotune_configs`; its usage is demonstrated at the end of Listing 2.

APPENDIX B LICENSE INFORMATION

TransNormerLLM-7B model is licensed under the Apache 2.0 License.
toy-retnet-1.3B model is licensed under the Apache 2.0 License.
LongBench is licensed under the MIT License.